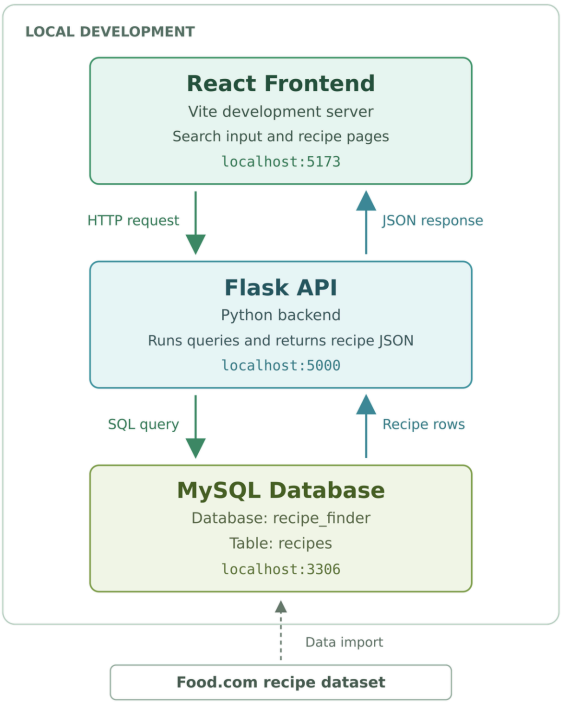
Product Document

Design

- Frontend: We use React and Ant Design for the user interface
- Backend: Python Flask handles API requests
- Database: MySQL to store recipe data



recipes — recipe_finder

COLUMN	TYPE	KEY
id	INT	PK
name	VARCHAR(255)	
description	TEXT	
ingredients	TEXT	
ingredients_raw_str	TEXT	
serving_size	VARCHAR(255)	
servings	INT	
steps	TEXT	
tags	TEXT	
search_terms	TEXT	

PK = Primary key
One row per recipe

Implementation

For the frontend implementation, we built a homepage to display available recipes, a search bar that allows users to find recipes by entering keywords, and a recipe detail page that displays information about the selected recipe.

The backend is implemented using Python Flask to handle API requests from the frontend. Based on the information provided in each API call, the corresponding function executes the query to fetch recipes from the database and return the result in JSON format.

The project uses MySQL to store recipe data in a table named `recipes`. The table contains the columns: `id`, `name`, `description`, `ingredients`, `ingredients_raw_str`, `serving_size`, `servings`, `steps`, `tags`, and `search_terms`. The primary key of this table is the recipe `id`.

Course Concepts

Table design:
The table contains columns for recipe attributes, including recipe `id`, `name`, `description`, `ingredients`, `ingredients_raw_str`, `serving_size`, `servings`, `steps`, `tags`, and `search_terms`.

Each recipe is identified using the `id`, which is the primary key. The columns use `INT`, `VARCHAR`, and `TEXT` data types.

```
CREATE TABLE recipes (
    id INT PRIMARY KEY,
    name VARCHAR(255),
    description TEXT,
    ingredients TEXT,
    ingredients_raw_str TEXT,
    serving_size VARCHAR(255),
    servings INT,
    steps TEXT,
    tags TEXT,
    search_terms TEXT
);
```

SQL:
The backend currently has two SQL queries to support the search feature and fetching individual recipe data.

```
if search_query:
   # Query database to search for name, ingredients, tags, and search terms of recipes
   sql_query = "SELECT * FROM recipes WHERE name LIKE %s OR ingredients LIKE %s OR
tags LIKE %s OR search_terms LIKE %s LIMIT 50"
...
else:
   # Default query if no search parameter provided
   # Gets first 50 recipes from database
   cursor.execute("SELECT * FROM recipes LIMIT 50")
```

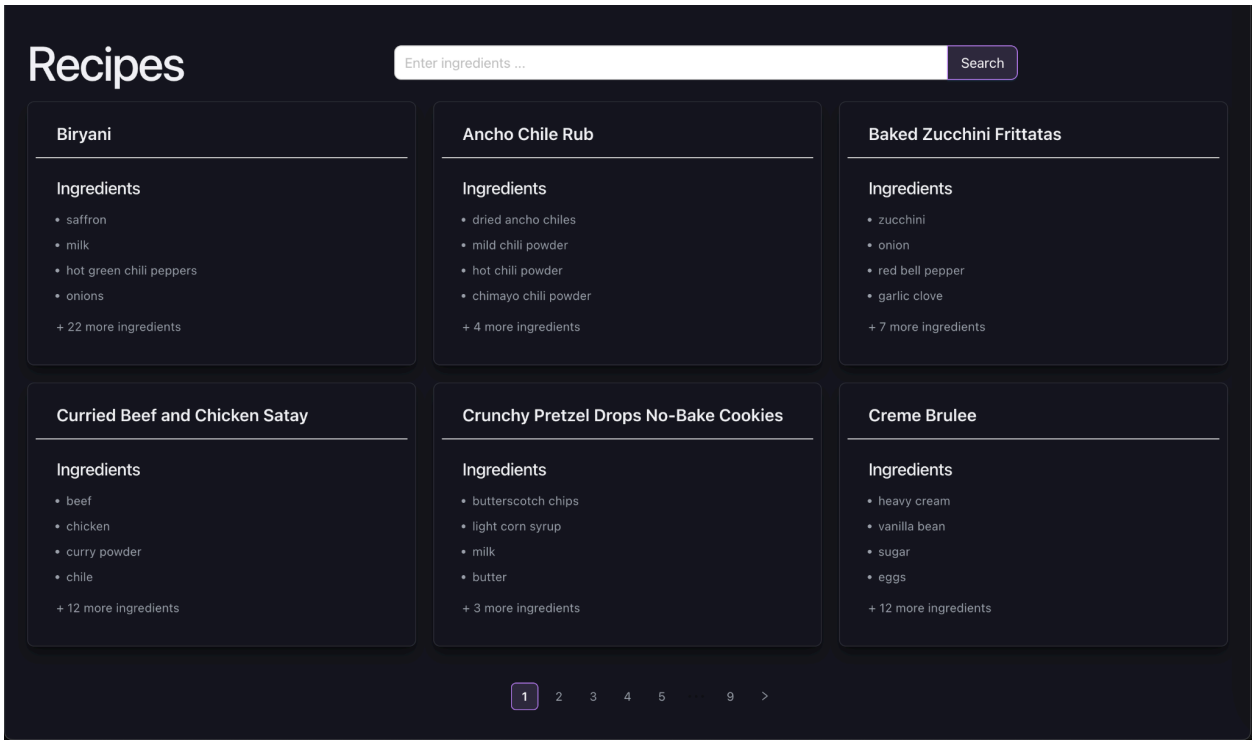
- The `SELECT` query with `WHERE` conditions search across the `name`, `ingredients`, `tags`, and `search_terms` columns. The `LIKE` operator is used for text matching. And the `OR` operator allows the application to find recipes that match any of the specified fields.
- This query returns the recipes that match the user's search input to the recipe `name`, `ingredients`, `tags`, or `search_terms`.
- When there is no search parameter, return all available recipes. Currently, we are limiting the output to the first 50 recipes for the prototype.

```
# Execute query to get all recipe fields
cursor.execute("SELECT * FROM recipes WHERE id = %s", (recipe_id,))
```

- This query returns an individual recipe based on the `id` of the selected item. When the user clicks on a recipe card, the frontend sends an API request containing the recipe's `id` to the backend to get the corresponding recipe details.

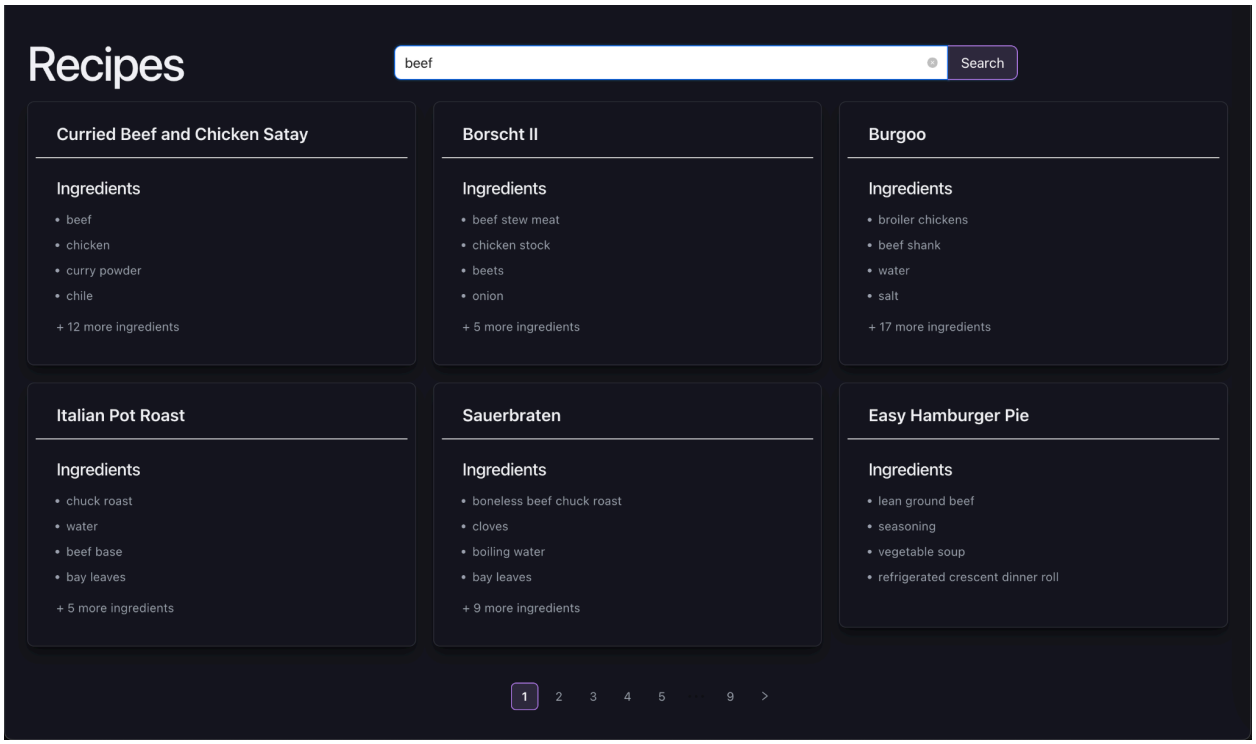
Prototype User Guide

1. Homepage



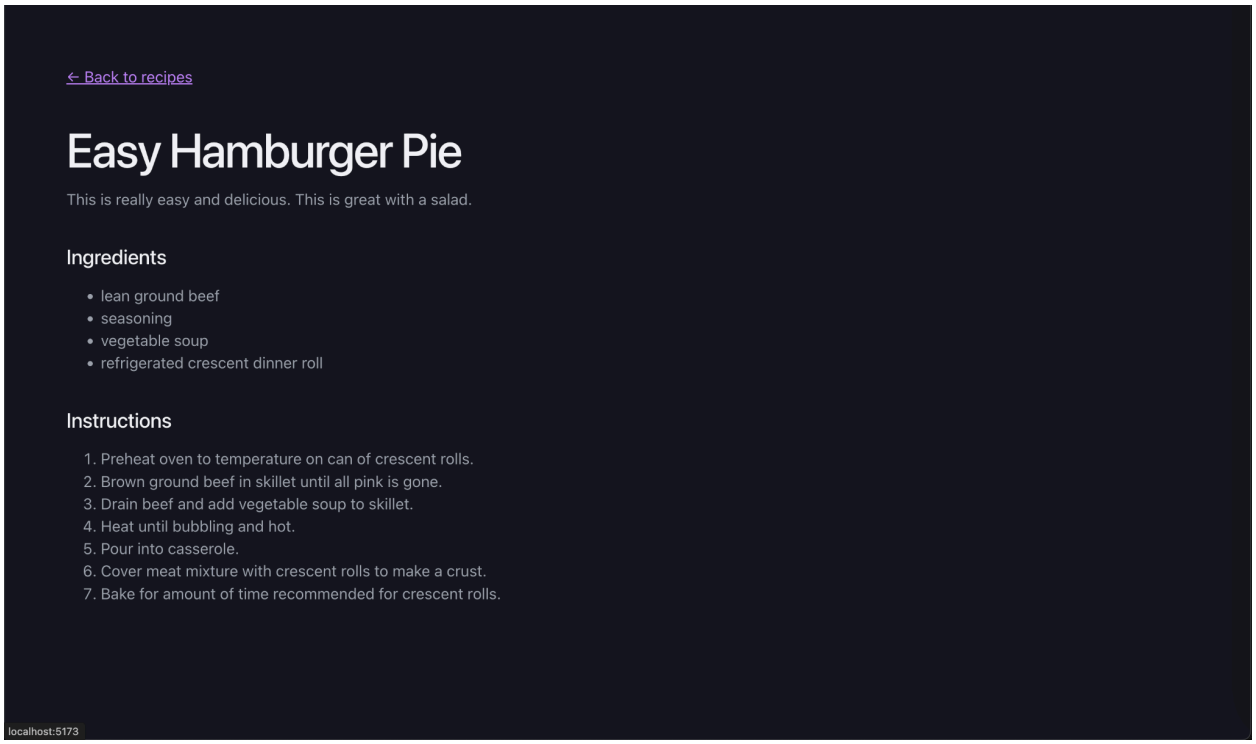
Homepage displays the available recipes when users first open the application

2. Search results for “beef”



Homepage displays recipes matching the input “beef”

3. Detail page for “Easy Hamburger Pie” recipe



Detail page displays information about the selected recipe